Kotlin

Kotlin Class *

Kotlin offers concise syntax for declaring classes. To declare a class, use the **class** keyword followed by the class name.

Classes provide blueprints from which objects can be constructed. An object is an instance of a class that consists of data specific to that object. You can use objects or class instances interchangeably.

As an analogy, imagine that you build a house. A class is similar to an architect's design plan, also known as a blueprint. The blueprint isn't the house; it's the instruction for how to build the house. The house is the actual thing, or object, which is built based on the blueprint.

Kotlin

Kotlin Class *

Just like the house blueprint specifies multiple rooms and each room has its own design and purpose, each class has its own design and purpose. To know how to design your classes, you need to get familiar with *object-oriented programming (OOP)*, a framework that teaches you to enclose data, logic, and behavior in objects.

Kotlin

Kotlin Class *

OOP helps you simplify complex, real-world problems into smaller objects. There are four basic concepts of OOP, each of which you learn more about later in this codelab:

- **Encapsulation.** Wraps the related properties and methods that perform action on those properties in a class. For example, consider your mobile phone. It encapsulates a camera, display, memory cards, and several other hardware and software components. You don't have to worry about how components are wired internally.
- **Abstraction.** An extension to encapsulation. The idea is to hide the internal implementation logic as much as possible. For example, to take a photo with your mobile phone, all you need to do is open the camera app, point your phone to the scene that you want to capture, and click a button to capture the photo. You don't need to know how the camera app is built or how the camera hardware on your mobile phone actually works. In short, the internal mechanics of the camera app and how a mobile camera captures the photos are abstracted to let you perform the tasks that matter.
- **Inheritance.** Enables you to build a class upon the characteristics and behavior of other classes by establishing a parent-child relationship. For example, there are different manufacturers who produce a variety of mobile devices that run Android OS, but the UI for each of the devices is different. In other words, the manufacturers inherit the Android OS feature and build their customizations on top of it.
- **Polymorphism.** The word is an adaptation of the Greek root *poly-*, which means many, and *-morphism*, which means forms. Polymorphism is the ability to use different objects in a single, common way. For example, when you connect a Bluetooth speaker to your mobile phone, the phone only needs to know that there's a device that can play audio over Bluetooth. However, there are a variety of Bluetooth speakers that you can choose from and your phone doesn't need to know how to work with each of them specifically.

Kotlin

Kotlin Class *

A class consists of three major parts:

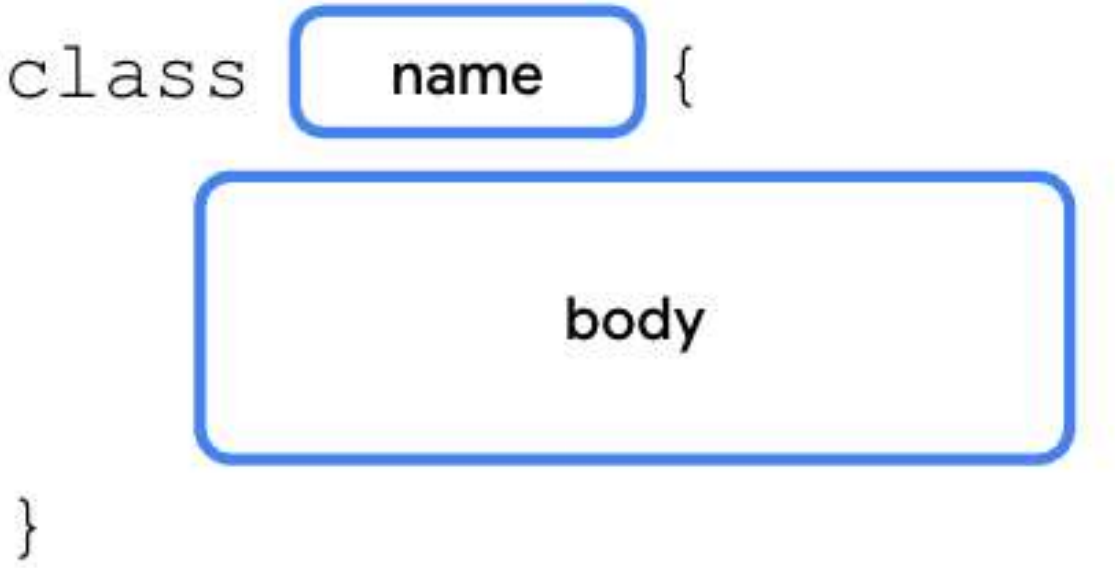
1. Properties. Variables that specify the attributes of the class's objects.

2. Methods. Functions that contain the class's behaviours and actions.

3. Constructors. A special member function that creates instances of the class throughout the program in which it's defined.

Kotlin

Kotlin Class *



Kotlin

Kotlin Class *

```
class MyClass {
 //empty body
}

fun main(){
}
```

Note: The class name is written in PascalCase, so each word begins with a capital letter and there are no spaces between the words. For example, in *S*martDevice, the first letter of each word is capitalized and there isn't a space between the words.

Kotlin

Kotlin Class *

Object: A **class** is like a blueprint for creating objects, just as a car company is used to make actual cars. An **object** is a real instance created from a class, holding data and performing actions. For example, if we have a Car class with properties like **brand** and **model**, we can create an object of that class to represent a real car. Objects help us organize and reuse code efficiently.

```
val name = ClassName()
```

```kotlin
class PhysicsCalculator(val version: String = "1.0") {

    // Speed: v = d / t
    fun calculateSpeed(distance: Double, time: Double): Double {
        return distance / time
    }

    // Distance: d = v * t
    fun calculateDistance(speed: Double, time: Double): Double {
        return speed * time
    }
}
```

```kotlin
class PhysicsCalculator(val version: String = "1.0") {

    // Speed: v = d / t
    fun calculateSpeed(distance: Double, time: Double): Double {
        return distance / time
    }

    // Distance: d = v * t
    fun calculateDistance(speed: Double, time: Double): Double {
        return speed * time
    }
}

fun main() {
    val obj = PhysicsCalculator()
    print(obj.calculateSpeed(120.0,2.0))

}
```

Kotlin

Kotlin Class *

Example code:

```
class SmartDevice {
    fun turnOn() {
        println("Smart device is turned on.")
    }

    fun turnOff() {
        println("Smart device is turned off.")
    }
}
```

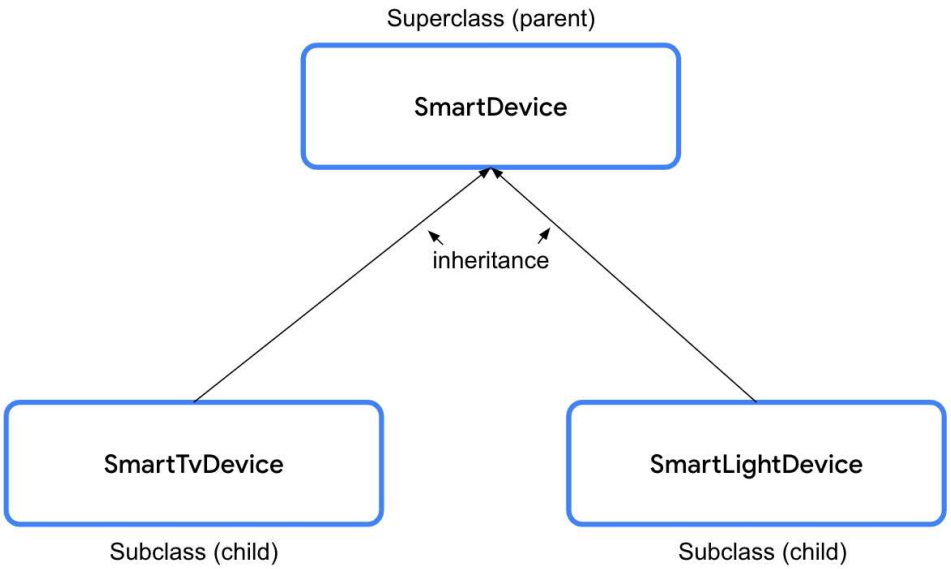
```
fun main() {
    val smartTvDevice = SmartDevice()
    smartTvDevice.turnOn()
    smartTvDevice.turnOff()
}
```

Here, **smartTvDevice** is the object of the **SmartDevice()** class

Kotlin

Kotlin Class * Inheritance

Inheritance lets you build a class upon the characteristics and behavior of another class. It's a powerful mechanism that helps you write reusable code and establish relationships between classes.



Inheritance Example

```
// Parent Class (marked as 'open' so others can inherit from it)
open class Device(val brand: String) {
    fun turnOn() {
        println("The $brand device is now ON.")
    }
}
```

Inheritance Example

```
// Parent Class (marked as 'open' so others can inherit from it)
open class Device(val brand: String) {
    fun turnOn() {
        println("The $brand device is now ON.")
    }
}

// Child Class (inherits from Device using the ':' symbol)
class Smartphone(brand: String, val os: String) : Device(brand) {
    fun takePhoto() {
        println("Taking a photo on the $os system...")
    }
}
```

Inheritance Example

```
// Parent Class (marked as 'open' so others can inherit from it)
open class Device(val brand: String) {
    fun turnOn() {
        println("The $brand device is now ON.")
    }
}

// Child Class (inherits from Device using the ':' symbol)
class Smartphone(brand: String, val os: String) : Device(brand) {
    fun takePhoto() {
        println("Taking a photo on the $os system...")
    }
}

fun main() {
    val myPhone = Smartphone("Samsung", "Android")

    // Using a method inherited from the parent
    myPhone.turnOn()

    // Using a method unique to the child
    myPhone.takePhoto()
}
```

Kotlin

Inheritance

Kotlin Class *

```
class SmartTvDevice(deviceName: String, deviceCategory: String) :
    SmartDevice(name = deviceName, category = deviceCategory) {

    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }

    var channelNumber = 1
        set(value) {
            if (value in 0..200) {
                field = value
            }
        }
}
```

My Application2 Version control Medium Phone API 36 app

MainActivity.kt

```kotlin
package com.example.myapplication2

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplication2Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

lyft
Expedia

println(“Thank You”)